

THE @KEREM.TECH FIELD GUIDE



BUG BOUNTY METHODOLOGY

The Complete Beginner's Field Guide

RECON



ENUMERATE



HUNT



REPORT

*“If it has an input,
it has a weakness.”*

SWIPE TO LEARN



What Is Bug Bounty?

Organizations invite ethical hackers to find security flaws in their systems, then reward them for every valid bug reported.



You Get Paid

Every unique, valid vulnerability earns a cash bounty, points or swag, scaled by its severity.



It Is Legal

A program's scope is your written permission to test those exact targets without any fear.



Responsible Disclosure

You report the flaw privately and give the team time to patch it before any public mention.



GOLDEN RULE

No permission, no testing. The scope is your contract. Never step outside it.

Programs & Platforms

Not every program pays the same way. Knowing the type tells you what to expect before you invest hours.



Paid Bounty

Valid bugs pay real cash, scaled by how severe and impactful the finding turns out to be.



Disclosure (VDP)

No cash on offer, but you still get legal safe harbour and public recognition for the work.



Public Programs

Open to every hunter, which means more competition and far faster duplicate reports.



Private Programs

Invite-only and much quieter, usually rewarding more per hour once you earn access.



HOW IT WORKS

Platforms are the middleman: they host the scope, route your reports and handle the payout.

The Hunter's Mindset

Tools only find what you tell them to. The real bugs are found by how you think.



Assume It Is Broken

Treat every input, button and feature as guilty until you have proven that it is actually safe.



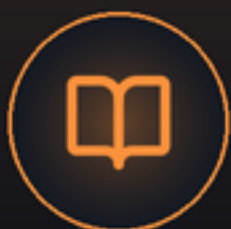
Follow Your Curiosity

Errors and strange behaviour are breadcrumbs. The interesting bug is usually just past them.



Be Relentless

The best findings sit deep, and most hunters give up long before they ever reach them.



Never Stop Learning

Every new framework ships fresh bugs, so read disclosed writeups and keep tinkering daily.

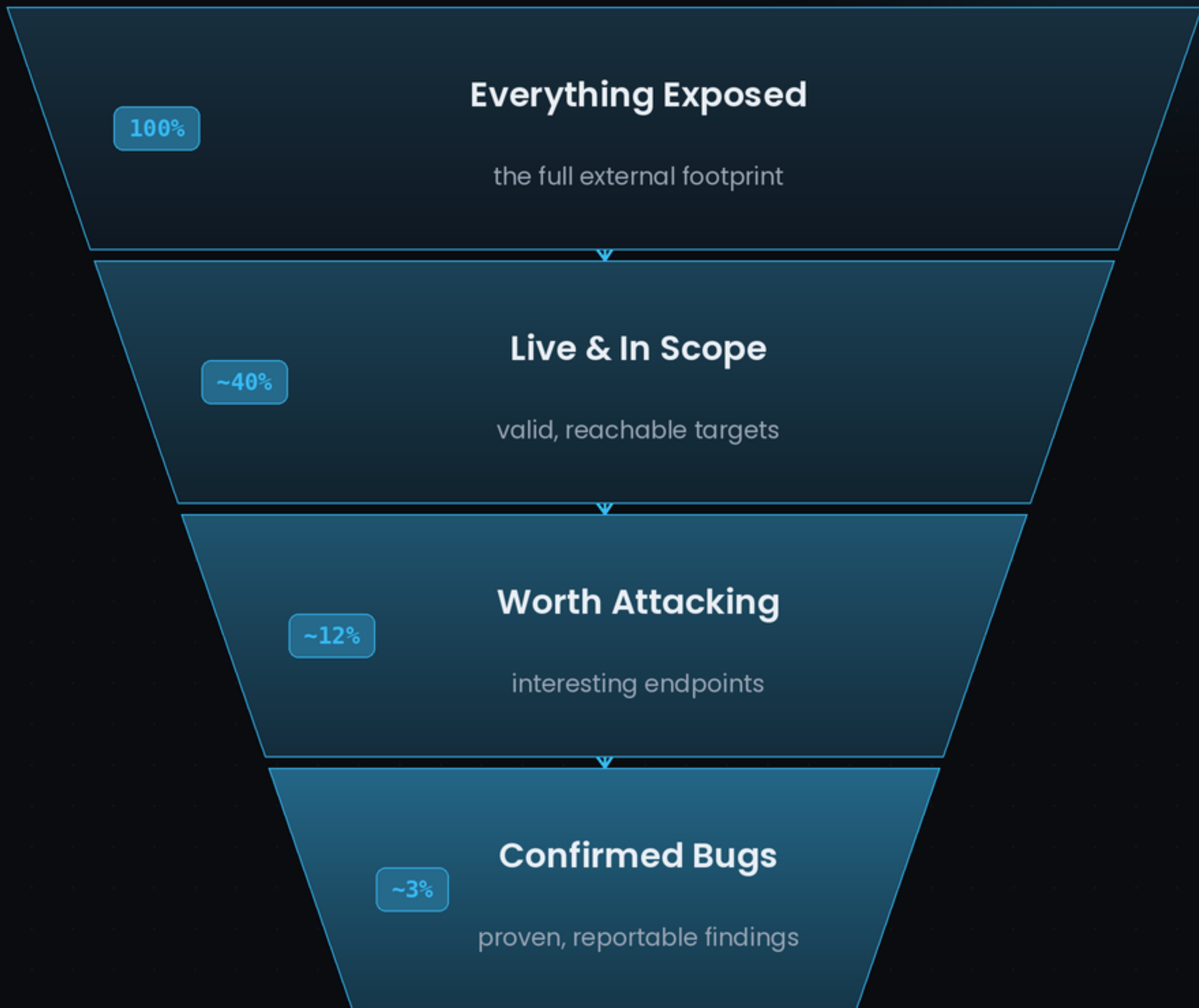


MINDSET

Curiosity is the one skill no scanner can ever replace. Cultivate it on every single target.

Why Follow A Method?

A method narrows a huge attack surface down to the few bugs that are real, in-scope and worth reporting.



WHY IT MATTERS

Coverage beats luck. A repeatable method means you never accidentally test the same way twice.

The Roadmap

Eight phases, from first look to payout. Each one feeds the next.

01

Scope & Rules

Define exactly what you may touch

02

Reconnaissance

Map the organization's footprint

03

Subdomain Enum

Expand the attack surface

04

Fingerprinting

Identify tech, services and versions

05

Content Discovery

Surface hidden endpoints and files

06

Map the Surface

Understand how the app behaves

07

Exploit & Validate

Prove real, measurable impact

08

Reporting

Write it up clearly and get paid

Scope & Rules

Read the scope twice before any testing. This is the single most important step of all.



In Scope

- **Listed domains & wildcards**
Plus any explicitly named asset
- **Named mobile & API targets**
Apps and endpoints written in scope
- **Accounts you create**
Test only on data you own



Out Of Scope

- **Anything not on the list**
If unsure, treat it as off-limits
- **DoS, DDoS & spam floods**
Never degrade availability
- **Phishing or staff tricks**
Humans are never in scope



REMEMBER

Rules also limit HOW you test. No automated scanners allowed?
Respect it to the letter.

Passive Recon

Gather intel without touching the target. No packets sent, no logs, no trace left behind.



WHOIS & DNS

Reveals the owner, name servers, mail records and IP ranges quietly tied to the org.



Search Dorking

Advanced search operators surface exposed files, logins and forgotten pages for free.



Cert Transparency

Public TLS certificate logs list subdomains the company never meant to advertise.



Public OSINT

Code repositories, job posts and old breaches quietly leak the internal tech stack.



PRO TIP

Passive recon is invisible and risk-free. Exhaust it before sending a single packet.

Subdomain Enum

More subdomains means more attack surface. Forgotten hosts are where the easy wins hide.



Passive Sources

Pull already-known subdomains from public datasets and cert logs, with zero noise.



Active Brute Force

Guess subdomain names from large wordlists, resolving each one against the DNS.



Permutations

Mutate the names you found, like dev, staging and api, to uncover even more hosts.



Resolve & Probe

Keep only the hosts that are alive and actually answer on the common web ports.



PRO TIP

Re-run enumeration over time. Fresh subdomains appear as the company keeps shipping.

Fingerprinting

Now you touch the targets. Identify what is running so you know exactly what to attack.



Port & Service Scan

Find which ports are open and identify the exact services listening behind each one.



Tech Fingerprinting

Detect the server, framework, CMS and library versions powering every target.



Screenshot Triage

Snapshot every live host to spot logins, dashboards and odd apps at a single glance.



Flag Old Versions

Outdated software maps straight to known, public exploits you can look up instantly.



PRO TIP

Version numbers are gold. An old build often has a public, ready-made exploit waiting.

Content Discovery

Apps hide far more than they link to. Brute-force the paths that nobody ever advertises.



Directories & Files

Fuzz for admin panels, backups, config files and anything left unlinked on the server.



Hidden Endpoints

Find the API routes and actions that the visible interface never calls directly.



Mine JavaScript

Front-end JS files quietly leak endpoints, parameters and sometimes hard-coded secrets.



Parameter Discovery

Uncover undocumented inputs that often skip the validation the main form enforces.



PRO TIP

One endpoint hidden in a JS file has started more big bug chains than any scanner ever has.

Map The Surface

Stop and understand the app like a power user before you start attacking it.



Roles & Permissions

Map exactly who can do what, then spend your time trying to cross every one of those lines.



Data & Auth Flows

Trace how a single request travels from your input all the way to the database and back.



Interesting Features

Uploads, redirects, payments and imports are complex, so they tend to break first.



Take Good Notes

Organised notes turn a dozen small, scattered clues into one high-impact bug chain.



MINDSET

The hunter who truly understands the app beats the one with more tools. Every single time.

Access & Logic Flaws

These need no fancy payloads, just logic. They make perfect first bugs for beginners.



IDOR

Change an ID in a request to read another user's private data.

LOOK FOR

`?id=123 → ?id=124`



Broken Auth

Reuse, guess or skip tokens to authenticate as someone else.

LOOK FOR

`weak / missing session checks`



Access Control

Reach admin-only actions from a normal, low-privilege account.

LOOK FOR

`hidden /admin routes`



Business Logic

Abuse intended features in completely unintended ways.

LOOK FOR

`negative price, infinite coupon`



START HERE

Access-control bugs are the most reported class for a reason. Start your hunt right here.

Injection & Web Flaws

Here you feed the app crafted input and watch it misbehave in your favour.



XSS

Inject script that executes inside another user's browser session.

LOOK FOR

`<script>` in reflected input



SQL Injection

Break out of a query to read or modify the backend database.

LOOK FOR

`' OR 1=1 --` in inputs



SSRF

Make the server request internal URLs it should never reach.

LOOK FOR

`url=` params, webhooks



Open Redirect

Bend a redirect to send victims to an attacker-owned page.

LOOK FOR

`next=` , `redirect=` params



VALIDATE

Always confirm impact. A reflected value is not a bug until it actually executes.

Exploit & Validate

Found something odd? Prove that it is real and measure the damage, safely.



Do This

- **Build a minimal PoC**
Just enough to prove the flaw exists
- **Show concrete impact**
Tie it to real-world damage
- **Stop once confirmed**
Resist the urge to dig deeper



Never Do

- **Dump or alter user data**
One row of your own is enough
- **Run destructive payloads**
No deletes, no mass writes
- **Pivot into the network**
Report the door, do not walk through



THE GOAL

Your PoC should convince a tired triager in thirty seconds. No more, no less.

The Report

A clear report gets fixed fast and paid well. A messy one gets closed as informative.

1

Title

Bug type, location and impact in one line

2

Summary

What it is, in two plain-English sentences

3

Steps to Reproduce

Numbered and exact, copy-paste ready

4

Proof of Concept

A screenshot, raw request or short video

5

Impact

What an attacker actually gains from it

6

Remediation

A concrete suggestion on how to fix it

The Essential Toolkit

You do not need many tools, just the right one for each phase of the hunt.



Recon & OSINT

Passively map the footprint

- **Amass** subdomain mapping
- **Subfinder** passive subs
- **crt.sh** cert logs
- **Shodan** exposed hosts



Scanning

Find open ports & services

- **Nmap** service scan
- **Naabu** fast ports
- **Masscan** mass scan
- **RustScan** speed layer



Content Discovery

Brute-force hidden paths

- **ffuf** web fuzzer
- **feroxbuster** recursive
- **gobuster** dir brute
- **dirsearch** path brute



Intercept Proxy

Inspect & tamper traffic

- **Burp Suite** the standard
- **Caido** lightweight
- **ZAP** free & open
- **mitmproxy** cli proxy



Web Recon

Probe & crawl live hosts

- **httpx** probe hosts
- **katana** crawler
- **gau** known urls
- **waybackurls** archive urls



Vuln Scanning

Template-based detection

- **Nuclei** templates
- **dalfox** xss scanner
- **sqlmap** sqli automation
- **nikto** server scan

Beginner Mistakes

Sidestep these four traps and you are already ahead of most newcomers.



Testing out of scope

→ Always read the scope first. Out-of-scope work earns zero reward and risks a ban.



Reporting with no impact

→ Show real, demonstrable damage, not just a value reflected back on the page.



Chasing easy duplicates

→ Hunt depth and freshly added assets, not the obvious low-hanging fruit everyone grabs.

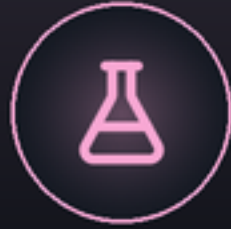


Vague, messy reports

→ Clean, exact reproduction steps get your finding triaged and paid for faster.

Where To Practice Next

Sharpen your skills legally, both before and alongside any real targets.



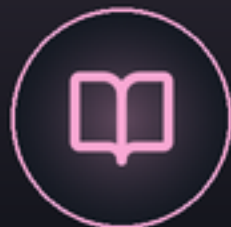
Vulnerable Labs

Practice on apps deliberately built to be hacked, so you can break things with zero risk.



Guided Academies

Free, structured web-security courses can take you from absolute zero to job-ready.



Read Disclosed Reports

Public writeups are a goldmine, teaching the exact methodology that earned real payouts.



Start On VDPs

Low-pressure, safe-harbour targets let you build a track record before chasing big bounties.



LAST TIP

Consistency wins. One focused hour a day beats one exhausting marathon a month.



SAVE THIS GUIDE

Your bug bounty playbook,
one screenshot away.



Instagram

@kerem.tech



X

@keraattin



GitHub

@keraattin



YouTube

Kerem Tech



Threads

@kerem.tech



Facebook

@kerem.tech

FOLLOW · SAVE · SHARE



@kerem.tech

@keraattin

